🟢 Aula 10 — Mini Projeto Vue.js (4h)

🎯 Objetivos da aula

- **Revisar** todos os conceitos do módulo Vue.js.
- Consolidar **arquitetura frontend profissional**.
- Implementar um **mini sistema CRUD** completo.
- Integrar **SPA + Vue Router + Pinia + API**.
- Realizar **avaliação prática** baseada em critérios técnicos.

---

1) Teoria — Revisão Geral (≈ 1h)

1.1 O que já dominamos

- **Componentização** (responsabilidade única)
- **Props / $emit** (fluxo unidirecional)
- **Computed / Watch** (reatividade correta)
- **Lifecycle Hooks** (quando executar o quê)
- **Vue Router** (SPA real)
- **Axios** (consumo de API)
- **Pinia** (estado global)

1.2 Organização de Projeto Frontend (padrão mercado)

Princípios:

- **Views** = páginas (ligadas a rotas)
- **Components** = UI reutilizável
- **Stores** = regras de negócio + estado
- **Services** = comunicação externa (API)
- **Router** = navegação centralizada

---

2) Prática — Mini Projeto CRUD (≈ 3h)

> 🎯 **Contexto do Projeto**
> Construir uma **SPA de gerenciamento de usuários** com:
>
> - **Listagem**
> - **Criação**
> - **Edição**
> - **Remoção**
> - **Detalhes**
>
> Fonte de dados: **JSONPlaceholder** (simulação REST)

2.1 Estrutura final do projeto

```
src/
 ├─ components/
 │    ├─ UserForm.vue
 │    └─ UserTable.vue
 ├─ views/
 │    ├─ HomeView.vue
 │    ├─ UsersView.vue
 │    └─ UserDetailView.vue
 ├─ stores/
 │    └─ userStore.js
 ├─ services/
 │    └─ api.js
```

```
├─ router/
│   └─ index.js
├─ App.vue
└─ main.js
```

2.2 Store — CRUD centralizado (Pinia)

src/stores/userStore.js

```
import { defineStore } from 'pinia' import api from '../services/api' export const useUserStore = defineStore('userStore', { state: () => ({ usuarios: [], loading: false, erro: '' }), getters: { total: (state) => state.usuarios.length, porId: (state) => (id) => state.usuarios.find(u => u.id === Number(id)) }, actions: { async carregar() { this.loading = true try { const res = await api.get('/users') this.usuarios = res.data } catch { this.erro = 'Erro ao carregar usuários' } finally { this.loading = false } }, adicionar(usuario) { this.usuarios.push({ id: Date.now(), ...usuario }) }, atualizar(usuarioAtualizado) { const index = this.usuarios.findIndex(u => u.id === usuarioAtualizado.id) if (index !== -1) this.usuarios[index] = usuarioAtualizado }, remover(id) { this.usuarios = this.usuarios.filter(u => u.id !== id) } } })
```

2.3 View — Lista + CRUD

views/UsersView.vue

```
<template> <section> <h2>Usuários</h2> <p>Total: {{ store.total }}</p> <UserForm :usuario-edicao="usuarioEdicao" @salvar="salvar" /> <UserTable :usuarios="store.usuarios" @editar="editar" @remover="store.remover" /> </section> </template> <script> import { useUserStore } from '../stores/userStore' import UserForm from '../components/UserForm.vue' import UserTable from '../components/UserTable.vue' export default { components: { UserForm, UserTable }, setup() { const store = useUserStore() store.carregar() return { store, usuarioEdicao: null } }, methods: { salvar(usuario) { usuario.id ? this.store.atualizar(usuario)
```

```
: this.store.adicionar(usuario) this.usuarioEdicao = null }, editar(usuario) { this.usuarioEdicao = { ...usuario } } } }
</script>
```

2.4 Componentes Reutilizáveis

components/UserForm.vue

```
<template> <form @submit.prevent="salvar"> <input v-model="form.name" placeholder="Nome" required /> <input v-model="form.email"
placeholder="Email" required /> <button>Salvar</button> </form> </template> <script> export default { props: { usuarioEdicao:
Object }, data() { return { form: { id: null, name: '', email: '' } } }, watch: { usuarioEdicao: { immediate: true, handler(u) {
if (u) this.form = { ...u } } } }, methods: { salvar() { this.$emit('salvar', { ...this.form }) this.form = { id: null, name: '',
email: '' } } } } </script>
```

---

components/UserTable.vue

```
<template> <table> <tr v-for="u in usuarios" :key="u.id"> <td>{{ u.name }}</td> <td>{{ u.email }}</td> <td> <button
@click="$emit('editar', u)">Editar</button> <button @click="$emit('remover', u.id)">Excluir</button> </td> </tr> </table>
</template> <script> export default { props: { usuarios: { type: Array, required: true } } } </script>
```

2.5 Router — SPA Completa

router/index.js

```
import { createRouter, createWebHistory } from 'vue-router' import HomeView from '../views/HomeView.vue' import UsersView from '../views/UsersView.vue' import UserDetailView from '../views/UserDetailView.vue' export default createRouter({ history: createWebHistory(), routes: [ { path: '/', component: HomeView }, { path: '/usuarios', component: UsersView }, { path: '/usuarios/:id', component: UserDetailView, props: true } ] })
```

---

3) Avaliação Prática (Rubrica)

🔍 Critérios (100 pontos)

Critério	Pontos
Estrutura de projeto	20
Uso correto do Pinia	20
CRUD funcional	25
Navegação SPA	15
Boas práticas (clean code)	20

✔️ **Aprovado:** ≥ 60 pontos
✔️ **Destaque:** ≥ 85 pontos

---

4) Checklist Final

- SPA funcionando sem reload
- CRUD completo

- Estado global centralizado
- Componentes reutilizáveis
- Código organizado e legível